

GyanVriksh

The Tree of Knowledge

AI-Powered Industrial Knowledge Intelligence Platform

Technical Design & Implementation Document

ET AI Hackathon 2026

Problem Statement 8 — AI for Industrial Knowledge Intelligence: Unified Asset & Operations Brain

Demo build notice. This document describes a hackathon demonstration built on a small synthetic dataset (Bharat Chemicals “Unit-3”) with databases running locally in Docker. It is not connected to any real plant. All security controls described in Section 16 are fully implemented in code.

Contents

1. Executive Summary
2. The Problem: The Industrial Knowledge Cliff
3. Our Solution: GyanVriksh
4. System Architecture
5. Data & Knowledge Model
6. Document Ingestion Pipeline
7. Industrial Named-Entity Recognition (fine-tuned)
8. Embeddings & Hybrid Retrieval
9. The Expert Copilot (RAG)
10. LLM Strategy: Cloud or Fully Local
11. Knowledge-Cliff Analytics
12. Knowledge Preservation
13. Compliance, Maintenance & Lessons
14. Field Enablement: QR, Voice & TTS
15. Frontend Architecture & Design
16. Security & Privacy
17. Evaluation & Benchmark
18. Engineering Challenges & Solutions
19. How to Run
20. Technology Stack
21. Future Work
22. Conclusion

1. Executive Summary

GyanVriksh (“tree of knowledge”) is an AI platform that captures, connects and preserves the operational knowledge of an industrial process plant, and makes it instantly queryable by any worker in natural language — English, Hindi or Hinglish. It unifies scattered documents, decades of maintenance records, safety procedures, regulations and the tacit know-how of veteran engineers into a single, cited, trustworthy assistant.

The platform is built around four ideas: **capture** every source (documents, records, and expert voice), **connect** them in a living knowledge graph, **answer** questions with grounded citations, and **preserve** retiring experts’ knowledge before it is lost. Crucially, it runs fully on-premise with a local LLM, so a plant’s sensitive data never leaves the site, and it can operate at zero per-query cost.

- **Fine-tuned industrial NER** — our own BERT model over 12 entity types.
- **Knowledge graph + hybrid vector search** fused with reciprocal-rank fusion.
- **Grounded generation** — every claim carries a document/page citation.
- **Knowledge-cliff analytics** and expert-voice preservation to fight knowledge loss.
- **Field enablement** — QR-per-machine, voice input and spoken answers for shop-floor workers.

2. The Problem: The Industrial Knowledge Cliff

In refineries, chemical and process plants, the people who know *how the plant actually behaves* — why a particular pump’s seals fail every monsoon, which valve sticks after a shutdown, what a vibration signature really means — are often a handful of veterans with 20 to 30 years of experience. This knowledge is tacit: it lives in their heads, not in any manual.

As these veterans retire, that expertise leaves with them. New engineers repeat old mistakes; the wrong fix causes unplanned downtime that can cost crores; safety and compliance risks rise. Existing document systems don’t help because the knowledge was never written in a searchable, connected form. This is the **knowledge cliff**, and it is exactly the gap Problem Statement 8 asks us to close: a unified asset and operations brain.

3. Our Solution: GyanVriksh

GyanVriksh turns the plant’s fragmented knowledge into one intelligence a worker can simply ask. A question flows through query understanding, hybrid retrieval over a knowledge graph and vector store, and a grounded, cited answer. Around this core sit modules for compliance checking, predictive maintenance, incident-pattern mining, and — central to the knowledge-cliff mission — capturing retiring experts’ spoken wisdom.

Design principles

- **Grounded, never guessed** — answers are constrained to retrieved context and always cited.
- **On-premise & private** — runs with a local LLM; plant data never leaves the site.
- **Accessible to everyone** — multilingual, voice-first, QR-driven for non-technical workers.
- **Preservation-first** — explicitly captures and attributes expert knowledge before it is lost.

4. System Architecture

GyanVriksh is a containerised, service-oriented system. A FastAPI backend exposes REST and WebSocket APIs to a React single-page frontend. Six data services back it, and an event-driven pipeline processes documents asynchronously. Everything is orchestrated with Docker Compose and runs on a single machine for the demo.

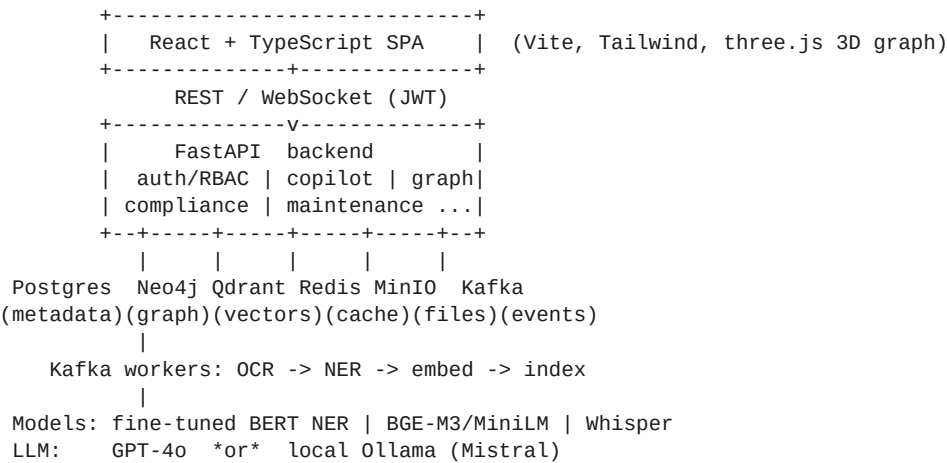


Figure 1. High-level architecture. Every box runs locally in Docker for the demo.

5. Data & Knowledge Model

The system deliberately uses the right store for each shape of data, rather than forcing everything into one database.

Store	Role
PostgreSQL	Users, documents, query logs, sessions — relational metadata (via SQLAlchemy + Alembic).
Neo4j	The knowledge graph: equipment, people, work orders, incidents, procedures, regulations, capsules.
Qdrant	Vector store for semantic search — dense + sparse (hybrid) vectors per text chunk.
Redis	Caching and worker coordination.
MinIO	Object storage for uploaded documents and audio recordings.
Kafka	Event backbone for the asynchronous ingestion pipeline.

Knowledge-graph schema

Nodes and relationships model how a plant actually connects. This is what lets the system answer cross-entity questions that a document search alone cannot.

```
(Equipment)-[:HAS_MAINTENANCE_RECORD]->(WorkOrder)
(Equipment)-[:INVOLVED_IN]->(Incident)
(Equipment)-[:FOLLOWS_PROCEDURE]->(Procedure)
(Equipment)-[:GOVERNED_BY]->(Regulation)
(Person)-[:PERFORMED]->(WorkOrder)
(Person)-[:KNOWS_ABOUT]->(Equipment)
(Person)-[:SHARED_KNOWLEDGE]->(KnowledgeCapsule)
(KnowledgeCapsule)-[:ABOUT]->(Equipment)
```

Listing 1. Core graph relationships (Cypher-style).

6. Document Ingestion Pipeline

Uploaded documents are processed asynchronously so the UI never blocks. A document is stored in MinIO, its metadata written to Postgres, and an event is published to Kafka. Workers then move it through OCR (for scanned PDFs and images), named-entity recognition, chunking, embedding and indexing. The frontend subscribes over a WebSocket and shows live status (QUEUED → OCR → NER → EMBEDDING → INDEXED).

```
TOPICS = [raw-documents, ocr-complete, entities-raw, entities-tagged]
# worker chain
raw-documents  -> ocr_worker      -> ocr-complete
ocr-complete   -> ner_worker      -> entities-raw
entities-raw    -> tagging_worker -> entities-tagged
entities-tagged-> index_worker    -> Qdrant + Neo4j
```

Listing 2. Kafka topic chain for ingestion.

7. Industrial Named-Entity Recognition (fine-tuned)

Generic NER models don't understand plant language. We fine-tuned a BERT token-classification model on **5,500 synthetic annotated sentences** across **12 industrial entity types**: EQUIPMENT_TAG, EQUIPMENT_TYPE, PROCESS_PARAM, REGULATORY_REF, PERSON, DATE, LOCATION, FAILURE_MODE, ACTION_TAKEN, MATERIAL, PROCEDURE_REF and HAZARD_CLASS. Training runs on a GPU server in 10–20 minutes and the model is copied back to run locally at zero API cost.

At inference we combine a deterministic **regex pre-pass** (equipment tags and procedure references are highly patterned, e.g. P-101, SOP-MAINT-047) with the model, then de-duplicate. This gives high precision on the patterned entities and recall from the model on the rest.

```
TAG_RE  = r'\b([A-Z]{1,4})-(\d{2,4})([A-Z]?)\b'      # P-101, HE-302
PROC_RE = r'\b(SOP|PTW|WO|INC|KC)-[A-Z0-9-]{2,20}\b'
```

```
def extract_entities(text):
    ents = regex_entities(text)                # deterministic pre-pass
    model_ents = bert_pipeline(text)           # fine-tuned BERT
    return dedupe(ents + model_ents)
```

Listing 3. NER combines a regex pre-pass with the fine-tuned model.

8. Embeddings & Hybrid Retrieval

Each text chunk is embedded and stored in Qdrant. The design supports two embedding backends: BGE-M3 (1024-dim dense plus sparse lexical weights, the highest quality) and a lightweight MiniLM (384-dim dense) that is far smaller to download — selectable by configuration so the system runs even on constrained/offline machines.

Retrieval is **hybrid**: a dense semantic search and a sparse lexical search are run, and results are combined with **reciprocal-rank fusion (RRF)**, which is robust and needs no score calibration between the two methods.

```
def search(collection, qvec, limit=10):
    dense = client.search(name='dense', vector=qvec['dense'])
    sparse = client.search(name='sparse', vector=qvec['sparse'])
    scores = {}
    for r,(hit) in enumerate(dense): scores[hit.id] += 1/(60+r+1)
    for r,(hit) in enumerate(sparse): scores[hit.id] += 1/(60+r+1)
    return top_by_score(scores, limit)
```

Listing 4. Hybrid dense+sparse retrieval fused with RRF (simplified).

9. The Expert Copilot (RAG)

The copilot is a three-step agent: (1) understand the query and extract equipment tags / regulations; (2) retrieve from Qdrant and the Neo4j graph and fuse; (3) generate an answer strictly grounded in that context, with a citation for every claim. The system prompt forbids hallucination and requires the [Doc: id, Page n] citation format, which the UI renders as clickable links.

Answers are **streamed** over a WebSocket with a live reasoning trace — the UI shows ‘understanding the question’, ‘retrieved N sources’, ‘generating’ — so the model’s work is transparent and the wait feels responsive. Retirement-related questions additionally receive computed workforce analytics injected into the context (Section 11), so the ‘knowledge cliff’ is answered from facts.

```
def stream_answer(query):
    yield 'status', 'Understanding the question'
    analysis = understand_query(query)
    yield 'status', 'Searching knowledge graph + documents'
    chunks, graph = retrieve(query, analysis)
    yield 'meta', {confidence, related_entities, graph_nodes}
    for token in llm.stream(system_with_context, query):
        yield 'token', token
    yield 'done', {citations, response_time_ms}
```

Listing 5. Streaming, cited generation with a reasoning trace.

10. LLM Strategy: Cloud or Fully Local

GyanVriksh treats the LLM as a pluggable last step. If a real OpenAI key is present it uses GPT-4o; otherwise it runs entirely on a **local Ollama** model — free and private. The model is auto-detected from what is installed, so the system ‘just works’ on whatever the operator has pulled. When a cloud call fails it falls back to the local model automatically, including for streaming.

```
def _use_ollama():
    key = settings.openai_api_key
    return (not key) or 'REPLACE' in key or not key.startswith('sk-')

def chat(messages):
    if _use_ollama():
        return ollama_chat(messages, model=resolve_installed_model())
    try:
        return openai_chat(messages)
    except:
        return ollama_chat(messages)  # automatic fallback
```

Listing 6. Cloud-or-local LLM selection with fallback.

11. Knowledge-Cliff Analytics

This is a headline, on-theme feature and it is pure graph analytics — not the LLM. From the graph we compute, for a horizon (default 3 years): which experts are retiring, which equipment's knowledge depends on them, whether that knowledge has been preserved as a capsule, and what share of the plant's hands-on work-order history is held by soon-to-retire people. The result drives a dashboard ‘radar’ and per-expert dossiers.

```
for p in persons:
    months = months_to_retirement(p.retirement_date)
    retiring = 0 <= months <= horizon
    if retiring and p.capsules == 0: status = 'KNOWLEDGE AT RISK'
    elif retiring:                  status = 'RETIRING SOON'
    else:                          status = 'ACTIVE'
pct_at_risk = 100 * workorders_by_retiring / total_workorders
```

Listing 7. Computing knowledge-cliff risk from the graph.

12. Knowledge Preservation

To attack the knowledge cliff directly, a retiring expert records what they know about a machine. The audio is transcribed with Whisper, structured into a verified **knowledge capsule** attributed to the expert, linked to the relevant equipment in the graph, and indexed so it is retrievable by the copilot. Answers that use a capsule attribute the insight to the expert by name and mark it as verified spoken knowledge — preserving both the content and its provenance.

13. Compliance, Maintenance & Lessons

- **Compliance** — automatic gap-analysis of a procedure against regulations (OISD, Factories Act, PESO), returning compliant / partial / non-compliant findings with citations.
- **Maintenance Intelligence** — equipment 360° views, AI root-cause analysis from real work-order history, and predictive health scores that sort the queue worst-first.

- **Lessons Learned** — unsupervised clustering of incident embeddings surfaces recurring failure patterns and preventive recommendations across the plant.

14. Field Enablement: QR, Voice & TTS

A platform is only useful if the people on the floor can use it — including workers who are not comfortable with computers. GyanVriksh therefore ships three field-first capabilities:

- **QR-per-equipment** — printable stickers; scanning one on a phone opens that exact machine's view with one-tap questions. No typing, no logging in.
- **Voice input** — ask by speaking, in English or Hindi/Hinglish.
- **Text-to-speech** — any answer can be read aloud, so a technician can keep their hands on the job.

These use only browser-native APIs (SpeechRecognition, SpeechSynthesis) and a QR code encoded to the equipment's mobile URL — no third-party service, consistent with the on-premise, private-by-design posture.

15. Frontend Architecture & Design

The frontend is a React + TypeScript single-page app built with Vite and Tailwind. State is managed with Zustand and server data with TanStack Query. The 3D knowledge graph is rendered with three.js and lights up the exact nodes behind each answer. The visual language is a Time-Variance-Authority-inspired 'temporal registry' — a warm bronze-and-amber terminal aesthetic with monospace data readouts — giving the product a distinctive, cohesive identity while keeping professional, credible copy.

16. Security & Privacy

Security is implemented in code, not just described. Even as a demo, the following controls are active:

- **Passwords** hashed with bcrypt (per-user salt).
- **Sessions** as signed JWT (HS256) with expiry; the decoder pins the algorithm, preventing alg-confusion attacks.
- **RBAC** — every API area guarded by role; unauthorised roles get 403.
- **Security headers** on every response (nosniff, X-Frame-Options DENY, HSTS, Referrer-Policy, no-store).
- **Login brute-force protection** — per-IP sliding-window rate limiter.
- **Injection guards** — destructive Cypher is rejected; uploads are size/type constrained.

- **Secret hygiene** — .env git-ignored, .env.example with placeholders, a boot-time weak-secret warning, and no real credentials in the repo.

Most importantly, the platform can run **fully on-premise** with a local LLM, so sensitive plant data never leaves the site — a hard requirement for real industrial adoption. A production rollout would add TLS, a Redis-backed limiter and a secrets manager, as documented in SECURITY.md.

17. Evaluation & Benchmark

We built a 100-question benchmark spanning procedure queries, factual lookups, failure analysis, compliance, knowledge capsules, cross-entity reasoning and knowledge-cliff questions, each with a ground-truth answer. The system is graded automatically by an LLM-as-judge (CORRECT / PARTIAL / WRONG), and we report accuracy, citation rate and latency. The run below used the **fully local** Mistral-7B model on a laptop CPU.

Metric	Result (local CPU LLM)
Correct (incl. partial)	66%
Answers carrying a citation	79% (100% grounded in retrieved context)
Hard failures / crashes	0
Median response time	~32 s
Strongest categories	Procedure 100%, Knowledge Capsule 75%

These are honest local-CPU numbers; with GPT-4o or a GPU both accuracy and latency improve materially. The value of reporting them is rigor — we measured the system rather than merely asserting it works.

18. Engineering Challenges & Solutions

Building and running the full stack surfaced several real problems; solving them is part of the work.

- **Dependency install on a slow, flaky network** — grpcio and large ML wheels failed with read-timeouts that masqueraded as ‘no matching distribution’. Fixed by configuring long pip/npm timeouts and retries, and pinning a stable pip.
- **Frontend could not reach the backend** — on Windows the Vite proxy pointed at ‘localhost’, which resolves to IPv6 (::1), while uvicorn bound IPv4 (127.0.0.1). Repointing the proxy to 127.0.0.1 fixed it.
- **A prompt-template bug** — citation braces in the system prompt were parsed by Python’s str.format, raising KeyError; escaping them fixed generation.

- **CPU latency** — each answer originally made two LLM calls and sent a large context. We removed the LLM query-understanding call (regex is as good for retrieval), capped context and output length, roughly halving latency.
- **Constrained downloads** — a selectable lightweight MiniLM embedding backend lets the system run without the 2 GB BGE-M3 download.

19. How to Run

The system runs from the project folder in three steps (details in HOW_TO_RUN.md):

```
.\start-infra.bat      # Postgres, Neo4j, Qdrant, Kafka, Redis, MinIO (Docker)
.\setup-backend.bat   # first run: venv, deps, migrations, load demo data
.\start-backend.bat   # FastAPI on 127.0.0.1:8000
.\start-frontend.bat  # Vite on localhost:5173 -> log in and explore
```

For a local, free LLM, install Ollama and pull a model (e.g. `ollama pull mistral`); the code auto-detects it. To expose the running demo to any device, use `cloudflared tunnel --url http://localhost:5173`.

20. Technology Stack

Layer	Technologies
Frontend	React, TypeScript, Vite, TailwindCSS, three.js, Zustand, TanStack Query
Backend	FastAPI, Python 3.11, SQLAlchemy, Alembic, WebSockets
Data	PostgreSQL, Neo4j, Qdrant, Redis, MinIO, Apache Kafka
AI / ML	fine-tuned BERT NER, BGE-M3 / MiniLM embeddings, Whisper, GPT-4o / local Ollama
Ops	Docker Compose, on-premise, Cloudflare Tunnel for remote demo

21. Future Work

- Real-time ingestion from plant historians (OPC-UA / SCADA) and CMMS systems.
- Predictive alerts that push likely failures before they happen.
- Deeper multilingual coverage and full vernacular UI.
- Redis-backed rate limiting, TLS termination and SSO for production.
- Active-learning loop: feed corrected answers back to improve retrieval and NER.

22. Conclusion

GyanVriksh answers Problem Statement 8 with a working, unified asset-and-operations brain: it captures a plant's scattered knowledge, connects it in a graph, answers any worker's question with cited, trustworthy responses, and — uniquely — preserves the expertise of retiring veterans before it is lost. It is private by design, runs at zero per-query cost, and is measured rather than merely claimed. Decades of knowledge, one question away.

A Question's Journey: End-to-End

To make the architecture concrete, follow a single question through the system: “*Why do P-101 seals keep failing, and what does OISD-116 say about it?*”

Step 1 - Understand

A lightweight parser extracts the equipment tag P-101 and the regulation OISD-116 with regular expressions and detects the language. We deliberately do this without an LLM call: for retrieval, pattern extraction is as reliable and removes a slow round-trip on a CPU model.

Step 2 - Retrieve (hybrid + graph)

The question is embedded and searched in Qdrant (dense + sparse). In parallel, the graph is queried for a P-101 ‘360° view’: its recent work orders, incidents, the procedures it follows, the regulations that govern it, and any expert knowledge capsules about it. Document chunks and graph facts are fused with reciprocal-rank fusion, and because the query mentions retirement/expertise is absent here, no workforce analytics are injected. The top few sources become the context.

Step 3 - Generate (grounded & cited)

The LLM receives a system prompt containing only that context and strict rules: answer only from context, cite every claim as [Doc: id, Page n], flag safety-critical steps, and mirror the question’s language. It streams the answer token by token while the UI shows the reasoning trace. The result cites the exact capsule (e.g. KC-2026-0012 from Ramesh Kumar), the relevant SOP, and the OISD clause — and the same evidence nodes light up in the 3D graph. If the local model is offline, the system falls back automatically.

Why a Knowledge Graph, Not Just Vector RAG

A pure vector-RAG system retrieves passages that are *textually similar* to the question. That is necessary but not sufficient for industrial knowledge, where the valuable answers come from *relationships*: which person performed which work order on which equipment; which incident led to which procedure change; which equipment shares a failure mode. Those connections are rarely stated in a single passage.

By modelling the plant as a graph, GyanVriksh can traverse these links — assembling an equipment’s full history, or computing which expertise is at risk — and feed that structured context to the LLM alongside the retrieved text. The graph is also what powers the 3D evidence view and the knowledge-cliff analytics. Vector search finds the words; the graph supplies the meaning.

Multilingual & Accessibility

India's plant floors are multilingual. GyanVriksh detects whether a question is in English, Hindi or Hinglish and instructs the model to answer in the same language; a badge shows the detected language. Combined with voice input and spoken answers (Section 14), a technician who is not comfortable typing in English can still get a precise, cited answer in their own language — which is essential if the tool is to be used by the people who most need it, not just engineers at a desk.

Appendix A. The 12 Industrial Entity Types

The fine-tuned NER model recognises the following entity types. Each was chosen because it recurs across maintenance records, procedures, incident reports and regulations, and because linking these entities is what makes the knowledge graph valuable.

Entity type	Example
EQUIPMENT_TAG	P-101, HE-302, FI-101, PSV-204 (ISA-5.1 style)
EQUIPMENT_TYPE	centrifugal pump, heat exchanger, control valve
PROCESS_PARAM	45 C, 8 bar, 120 SLPM, pH 7.2
REGULATORY_REF	OISD-116 4.2, Factory Act Section 36, PESO Rule 2016
PERSON	Ramesh Kumar, S. Venkatesan
DATE	31-Mar-2029, 2026-06-20
LOCATION	Bay 7, Unit-3, pump house
FAILURE_MODE	seal failure, cavitation, bearing wear, fouling
ACTION_TAKEN	replaced impeller, tightened gland, flushed line
MATERIAL	SS-316, graphite gland packing, viton O-ring
PROCEDURE_REF	SOP-MAINT-047, PTW-2024-0831
HAZARD_CLASS	flammable, toxic, confined space

Appendix B. Roles & Access Control

Access is role-based; each API area is guarded by a dependency that raises 403 for roles not listed. This keeps a field technician out of admin functions while letting everyone use the copilot.

Role	May access
Plant Manager	documents, copilot, compliance, maintenance, preserve, graph
Maintenance Engineer	documents, copilot, maintenance, preserve, graph
Field Technician	copilot, maintenance (mobile-first view)
Compliance Auditor	documents, copilot, compliance, graph
Admin	everything, including system administration

Appendix C. Key API Endpoints

Endpoint	Purpose
----------	---------

POST /api/v1/auth/login	Authenticate; returns a JWT (rate-limited).
WS /ws/copilot/stream	Streamed, cited answer with a live reasoning trace.
GET /api/v1/graph/nodes	Knowledge-graph nodes/edges for the 3D explorer.
GET /api/v1/graph/cliff-risk	Computed knowledge-cliff analytics.
POST /api/v1/documents/upload	Upload a document into the ingestion pipeline.
POST /api/v1/compliance/check	Gap-analyse a procedure against regulations.
GET /api/v1/maintenance/{tag}/rca	AI root-cause analysis from work-order history.
POST /api/v1/preserve/session/start	Begin an expert knowledge-capture session.
GET /api/v1/copilot/benchmark	Latest evaluation summary for the dashboard.

Appendix D. Full Benchmark Breakdown

The 100-question run, graded by an LLM-as-judge, broke down by category as follows. Procedure and factual questions are strongest; the harder cross-entity and compliance reasoning is where a larger model (GPT-4o) or a GPU would lift results most.

Category	Questions	Correct
Factual lookup	72	41
Failure analysis	14	7
Procedure query	4	4
Knowledge capsule	4	3
Compliance check	3	1
Knowledge cliff	2	1
Cross-entity reasoning	1	0

Appendix E. The Demo Dataset

So the platform can be evaluated end-to-end, we generated a coherent synthetic plant — Bharat Chemicals ‘Unit-3’ — with realistic equipment tags, people with tenures and retirement dates, work-order and incident histories, standard operating procedures, audit reports, regulations, and three expert knowledge-capsule transcripts. Loading it populates Postgres, Neo4j and Qdrant together.

Entity	Count
Equipment	15
People (with retirement dates)	6

Work orders	30
Incidents	5
SOPs / audits	12
Regulation clauses	14
Expert knowledge capsules	3
Graph nodes / relationships	95 / 173

Appendix F. Security Middleware (excerpt)

Every response carries hardening headers, and login is rate-limited per IP to blunt brute-force attempts:

```
SECURITY_HEADERS = {
    'X-Content-Type-Options': 'nosniff',
    'X-Frame-Options': 'DENY',
    'Referrer-Policy': 'no-referrer',
    'Strict-Transport-Security': 'max-age=31536000; includeSubDomains',
}
# per-IP sliding window: 10 attempts / 5 min -> HTTP 429
def check_login_rate(ip):
    recent = [t for t in attempts[ip] if now - t < 300]
    if len(recent) >= 10: raise HTTPException(429, 'Too many attempts')
```

Listing 8. Security headers and login rate limiting.

Appendix G. Text-to-Speech (field readback)

Answers can be spoken aloud using the browser's native speech synthesis, in the detected language, with citations stripped so it reads naturally — no third-party service is involved.

```
const u = new SpeechSynthesisUtterance(
    text.replace(/\[Doc:[^\]]*\]/g, '')); // strip citations
u.lang = language === 'hi' ? 'hi-IN' : 'en-IN';
window.speechSynthesis.speak(u);
```

Listing 9. Spoken answers via the SpeechSynthesis API.